



UNIVERSITY OF
WATERLOO

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE-124 LAB MANUAL

V4.1A

LAB 4

Course: ECE-124 Digital Circuits and Systems

1 LAB4: VHDL for Sequential Circuits – Sequential Logic & State-Machines

Lab4 will be the first one that uses Sequential Logic. Background information on sequential logic will be briefly covered to suit the Lab4 requirements. Design elements from previous labs will be used for Lab4. Both Moore and Mealy State Machine designs will be in the design goals for this lab.

1.1 Lab4 Intended Learning Outcomes

By the end of this lab, students should be able to:

- 1) **UNDERSTAND** State Machine design concepts, including Moore or Mealy forms.
- 2) **UNDERSTAND** the problems creating register “metastability”
- 3) **LEARNING** the register **SYNCHRONOUS DESGN** technique.

1.2 Lab4 Outline

The following new topics will be presented:

1. What are State Machines?
2. Some VHDL Implementations of State Machine sections
3. What is Metastability?
4. Using Synchronous Design techniques to Eliminate Metastability
5. Project Brief for Lab4

1.3 Lab4 Activities

1.3.1 Recall from Lab3

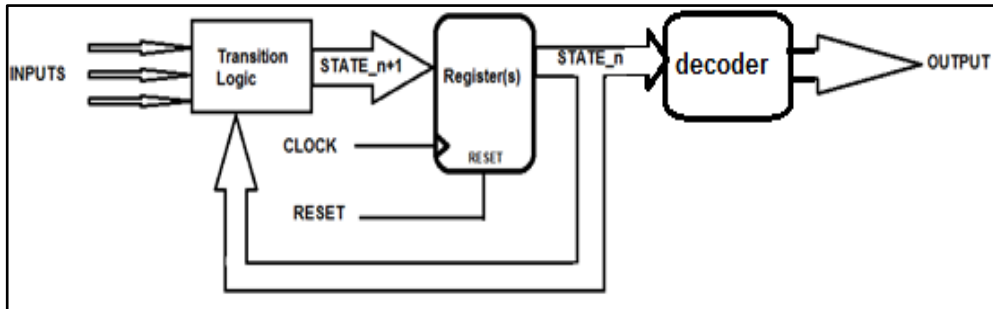
Last time in the lab, we developed a 4-bit Magnitude Comparator. The comparison function was implemented in two levels. This technique often simplifies the creation phase of a design. This Magnitude Comparator function was used in an Energy Monitoring Controller to determine current and desired temperature differences. In Lab3, the topic of Behavioral VHDL (Process construct) was introduced and a couple of separate examples were covered for development. The FIRST example of the Behavioural form used in Lab 3, was for Combinational Logic (M-C Tester). The SECOND example was for developing some Sequential Logic (for the HVAC). Lab3 also had some exercises for creating a bi-directional Shift Register and an up/down counter and checking their operations in simulations.

1.3.2 What is a State Machine?

A sequence of logic processing steps in an algorithm can be implemented with a STATE MACHINE design. State Machines can be used to provide a systematic approach to designing a logic solution for a digital function.

In a state machine design, the conditions that make it change from the CURRENT state to the NEXT state are various. Combinational logic is used to determine if or when those state transitions should occur. The state changes occur in synchronization with a state machine clock because registers are involved.

There are two primary classes of state machines used in sequential logic design: Moore and Mealy. To remember the names just think of two engineers sitting at a Thanksgiving table. One engineer politely says to the other “Would you like some Moore corn?” to which the second engineer replies, “Only if it’s not Mealy”. (Groan!).

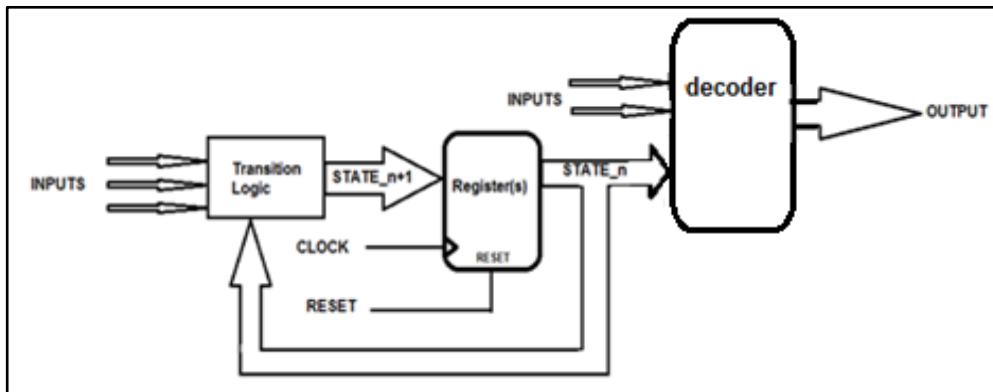


MOORE STATE MACHINE:

Output is a function of present state ONLY.

Moore SM's outputs are only changed when the state machine is clocked.

For any change of an INPUT to a MOORE State Machine an output change will NOT happen until after the next CLOCK CYCLE.



MEALY STATE MACHINE:

Outputs are functions of present state AND some inputs.

For any change of an INPUT to a MEALY State Machine, an output change MAY happen immediately or it might happen after the next clock cycle.

For this lab, either of the Moore and Mealy state machines can be used. Referring to two figures above, there are generally just three

sections in each state machine design. These are the Register, Transition and Decoder sections. Only the Register section is implemented as a Sequential Logic block. The Transition and Decoder sections are implemented as Combinational Logic blocks.

In all cases, Outputs from a STATE MACHINE only come from the DECODER section.

1.3.2.1 VHDL Implementations of Moore and Mealy State Machines

Provided in the Lab4 download files from Learn there is a VHDL file named "STATE_MACHINE_EXAMPLE". The Entity section for this file is shown below. It consists of a clock input, a reset input, a few transition controlling inputs and two outputs to signal when the state machine is in specific states.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

Entity State_Machine_Example IS Port
(
  clk_input, reset, I0, I1, I2          : IN std_logic;
  output1, output2                     : OUT std_logic
);
END ENTITY;
```

Further in the same State_Machine_Example file, in the Architecture declaration section, a list of states for a state machine with a TYPE statement named STATE_NAMES. This gives the state machine the set of state values for its processing. For your state names, please use meaningful names.

```
Architecture SM of State_Machine_Example is

  TYPE STATE_NAMES IS (s0, s1, s2, s3, s4, s5, s6, s7);  -- list all the STATE_NAMES values
  SIGNAL current_state, next_state : STATE_NAMES;        -- signals of type STATE_NAMES

BEGIN
```

Below the TYPE statement, two signals are declared to HOLD two separate values of the TYPE called STATE_NAMES to keep the “current” state and “next” state assignments for the state machine.

As mentioned previously, there are three sections of a State Machine. For this lab, they will be created using the VHDL Process construct for each section.

The Register section uses the state machine clock input and usually a RESET input signal in its Sensitivity List. These are used to advance the state machine to a new state in alignment with clock edge or to RESET it. This first state machine process is in a section shown as “Register_Section”.

IT IS SEQUENTIAL LOGIC because registers are created (or inferred) with this process construct.

```

-----
--State Machine:
-----

-- REGISTER_LOGIC PROCESS:
Register_Section: PROCESS (clk_input) -- this process synchronizes the activity to a clock
BEGIN
  IF(rising_edge(clk_input)) THEN
    IF (reset = '1') THEN
      current_state <= S0;
    ELSE
      current_state <= next_State;
    END IF;
  END IF;
END PROCESS;

```

For Lab 4, there is a clock enable to be added to the Register section of the state machine to advance the state at periodic intervals. (Recall how to do that from Lab3).

The Transition section (described in a second Process construct) uses only combinational logic and it is typically implemented with the VHDL “CASE” statement.

```

case <expression> is
  when <constant_expression> =>
    -- Sequential Statement(s)
  when <constant_expression> =>
    -- Sequential Statement(s)
  when others =>
    -- Sequential Statement(s)
end case;

```

It's Process construct has a Sensitivity List some State Machine inputs I0, I1, I2 etc. and the CURRENT State input coming from the Register section output.

A typical example of a CASE statement is shown at the left. NOTE: All “constant_expression” options in a VHDL case statement must be constant and unique. In addition, the case statement entries must include a “when others” clause at the end if not all combinations of “constant_expressions” are defined.

The CASE statement construct can specify a signal group (for example: Pb_n(3 downto 0) etc.) for the “expression” field).

```

-- TRANSITION LOGIC PROCESS
Transition_Section: PROCESS (I0, I1, I2, current_state)
BEGIN
    CASE current_state IS
        WHEN S0 =>
            IF (I0='1') THEN
                next_state <= S1;
            ELSE
                next_state <= S0;
            END IF;
        WHEN S1 =>
            next_state <= S2;
        WHEN S2 =>
            IF (I0='1') THEN
                next_state <= S6;
            ELSIF (I1='1') THEN
                next_state <= S3;
            ELSE
                next_state <= S2;
            END IF;
        WHEN S3 =>
            IF (I0='1') THEN
                next_state <= S4;
            ELSE
                next_state <= S3;
            END IF;
        WHEN S4 =>
            next_state <= S5;
        WHEN S5 =>
            next_state <= S6;
        WHEN S6 =>
            IF (I0='1') THEN
                next_state <= S7;
            ELSE
                next_state <= S6;
            END IF;
        WHEN S7 =>
            IF (I2='1') THEN
                next_state <= S0;
            ELSE
                next_state <= S7;
            END IF;
        WHEN OTHERS =>
            next_state <= S0;
    END CASE;
END PROCESS;

```

An example of a Transition section being shown here on the left, has a CASE statement that is within the process labelled as “Transition_Section”. IT MUST BE COMBINATIONAL LOGIC. This is because no latches or registers are allowed (through inference). ALL sequential statements must include all outcomes for the IF statements (ELSE CLAUSES PRESENT etc.)

Note how specific inputs are observed by the Transition Logic only in specific “current” states and also how these inputs can cause the State Machine Transition section to generate the “next” state values.

For example, if an “I0” input signal is active when the State Machine Example “current” state is State “S2” the State Machine Transition section will force a jump of the State Machine to state “S6” as its “next” state.

A third section of the State Machine Example design, is the Decoder section (shown on the left below) and it sets the State Machine outputs “output1”, “output2” signals when the state machine state reaches specific states. It is a requirement that all the output signal levels are defined for each and every current_state value. This could be a Decoder section for a Moore State Machine design since the outputs are dependent ONLY on the state machine “current_state” values. The other figure on the right below shows an abbreviated version.

```
-- DECODER SECTION PROCESS for Moore
Moore_Decoder_Section: PROCESS (current_state)
BEGIN
  CASE current_state IS
    WHEN S0 =>
      output1 <= '1';
      output2 <= '0';

    WHEN S1 =>
      output1 <= '0';
      output2 <= '0';

    WHEN S2 =>
      output1 <= '0';
      output2 <= '0';

    WHEN S3 =>
      output1 <= '0';
      output2 <= '0';

    WHEN S4 =>
      output1 <= '0';
      output2 <= '0';

    WHEN S5 =>
      output1 <= '0';
      output2 <= '0';

    WHEN S6 =>
      output1 <= '0';
      output2 <= '1';

    WHEN S7 =>
      output1 <= '0';
      output2 <= '0';

    WHEN OTHERS =>
      output1 <= '0';
      output2 <= '0';
  END CASE;
END PROCESS;
```

```
-- DECODER SECTION PROCESS for Moore
Moore_Decoder_Section: PROCESS (current_state)
BEGIN
  CASE current_state IS
    WHEN S0 =>
      output1 <= '1';
      output2 <= '0';

    WHEN S6 =>
      output1 <= '0';
      output2 <= '1';

    WHEN OTHERS =>
      output1 <= '0';
      output2 <= '0';
  END CASE;
END PROCESS;
```

The Decoder section MUST BE COMBINATIONAL LOGIC because no latches are to be created (inferred).

For a Mealy State Machine, the major difference is with the Decoder section. For it, the outputs depend on the state value AND some inputs.

The Mealy State Machine state is like a “gate control” for an input signal to an output signal activation kind of operation. If we were to convert the previous Moore State Machine into a Mealy State Machine, then the following changes would happen to the Decoder section as shown in the left figure. Again, this section is COMBINATIONAL Logic.

To develop an example of the Mealy State Machine, let’s assume that the Transition Logic is still the same as in the Moore example earlier.

```
--Decoder Section PROCESS for Mealy
Mealy_DECODER_SECTION: PROCESS(I0, current_state)
BEGIN
  IF (current_state = S0) THEN
    output1 <= I0;
    output2 <= '0';
  ELSIF (current_state = S6) THEN
    output1 <= '0';
    output2 <= I0;
  ELSE
    output1 <= '0';
    output2 <= '0';
  END IF;
END PROCESS;
```

In the above example, output1 and output2 are combinational outputs driven by the input I0 (**NOTE: The input I0 must be INCLUDED in the SENSITIVITY LIST of the Process construct for this type of Decoder**). But I0 is allowed to activate those outputs only when the state machine is in state S0 (for output1) or state S6 (for output2). If required, the outputs can be driven during numerous different states with the appropriate inputs being considered.

The type of state machine to be used for an application, depends on the requirements of the application.

Further Insights:

Notice that for Mealy State Machines (with the outputs also being driven by combinational inputs), the outputs can suffer from asynchronous transitions of the inputs. For example, if I0 is “intermittent” then output1 and output2 will be “intermittent” as well during the enabling states. This is the main disadvantage of Mealy State Machines as compared to Moore State Machines. The advantage for Mealy State Machines is that often they can be designed with a fewer number of states than a Moore version. They may also change the outputs sooner since a state change may not have to occur before an output becomes active.

Alternatively, if using a Moore State Machine, the output1 would be driven to a ‘1’, BUT only during the current_state of “S0”. Outside of the S0 state, the direct input connection from “I0” to the outputs would be moot. The advantage that Moore State Machines over their Mealy State Machine counterparts is that external inputs are prevented from influencing the State Machine outputs directly. The State Machine must change “state” first and then the outputs are driven by decoding

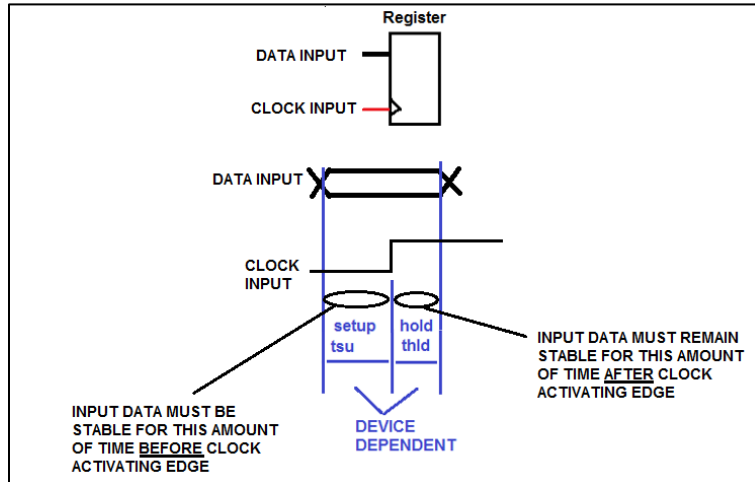
The code for STATE_MACHINE_Example given earlier, gives you a starting point for a typical State Machine. For the Lab4 project, you can make the modifications to make your state machine operate as a Moore or Mealy design. You can then declare it as a component at the top level VHDL file and then create an instance for it like the other functions.

NOTE 1: For any IF Statement or CASE Statement, any signals present in one clause must be present and defined in EVERY OTHER Clause, otherwise the VHDL compiler will infer latches.

NOTE 2: Outputs from a State Machines ONLY come from the DECODER Section

1.3.3 What is Metastability?

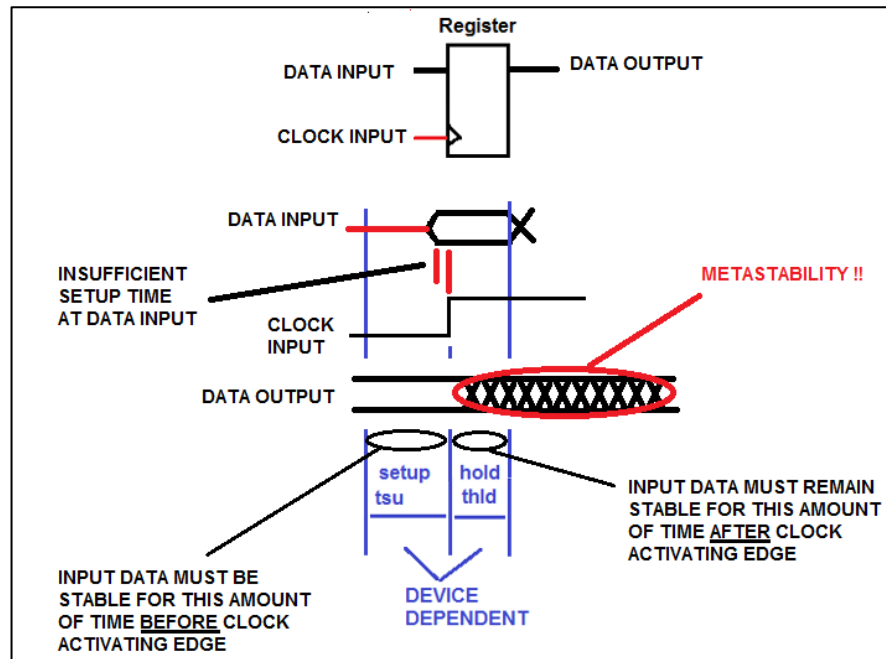
Every register has some basic timing constraints at its Data input known as “Setup time (tsu)” and “Hold time (thld)”. These constraints are expressed as relative to the activating edge of the register clock input. Refer to the figure below.



The Setup time (t_{su}) is the amount of time required for the data to be stable at the register input BEFORE the clock latching edge.

The Hold time (t_{hd}) is the amount of time required for the data to be stable at the register input AFTER the clock latching edge.

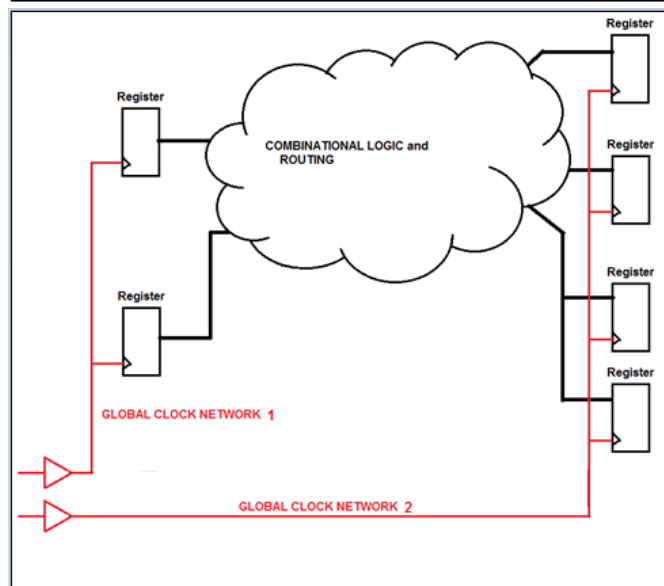
Violating these constraints can cause a problem at the register output. The figure below shows what happens to a register output when these timing constraints are not met. The “late arrival” of a Data Input relative to the register clock causes the Setup time violation. This result can be caused by an upstream signal delay due to signal routing in the device or by having too many levels of combinational logic to propagate through before the signal reaches the register.



This situation at the output is called METASTABILITY and it can lead to all sorts of undefined and weird behaviours of downstream digital logic. The output drifts between the two logic levels for some amount of time. If other logic uses that output for logic operations, things can quickly get undefined.

Registers also have a timing property aligned with its output known as "Clock to Output time". The "tco" is the time delay between the clock arriving at a register and the changing of the output of the register value. This aspect also has a role in the outcome of metastability issues with downstream registers.

But a BIG source of metastability problems in digital logic pipelines can come from employing multiple clock domains. These clock domains are asynchronous to each other (see Figure 92). Data outputs clocked registers from one clock domain may reach a pipeline register input that is running from a different clock domain.



Looking at the figure on the left, one can see that two register pipeline groups are connected (through the combinational logic and routing resources) but they run on different and **ASYNCHRONOUS** clock networks. For this situation the "robustness" of the design cannot be guaranteed. The operation is largely unpredictable.

There are techniques that allow the crossing of data and control signals between different clock domains but are beyond the scope of the course currently. For this course just one global clock source is to be considered for common clocking.

1.3.4 Using the Synchronous Design Techniques to Eliminate Metastability:

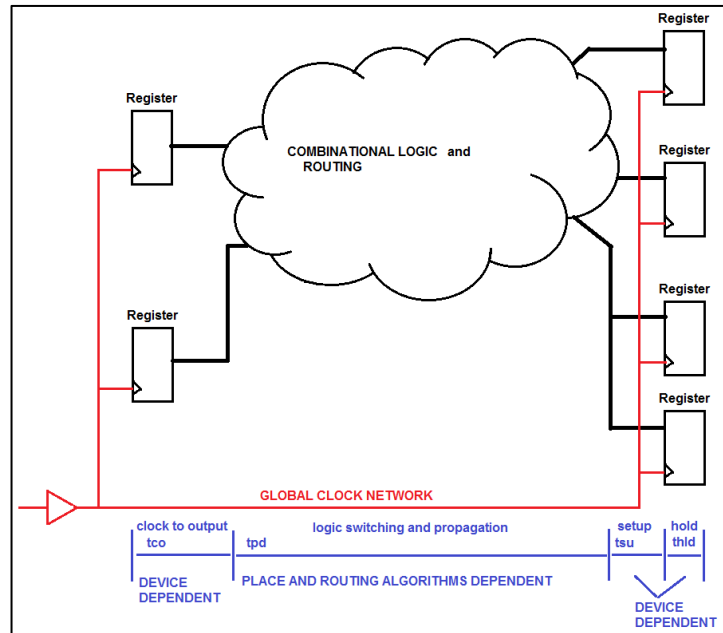
FPGA technologies have massive quantities of internal registers for state machines, counters, processors and all of them will use clock networking to run them. Between the registers, are equally huge numbers of combinational logic resources that do the digital logic transformations along the way.

To mitigate the possibility of METASTABILITY, an FPGA designer should take care in the arrangement of register pipeline designs. Use a common GLOBAL clock source as much as possible. This approach is usually referred to as “**Synchronous Design**”.

In digital design work, there are several design constraints that we must consider in order to have the design be robust and operate with high performance reliability.

This Lab will employ the concept of “synchronous design” in mind. Metastability problems generally “show up” when synchronous design techniques are not used (i.e.: Asynchronous design). This can happen when pipeline registers are not using the same clock source. It might be good enough for VERY SIMPLE and SLOW circuits but is generally not acceptable for designs that will go into a production setting.

For all FPGA design situations, it is strongly recommended to follow a “best practices” approach to designing the clocking networks. This approach uses **Synchronous Design** clocking (shown in the figure below). This is done by transferring the data and control signals in the various register pipelines using a **COMMON**, high-performance (low skew) global clock network. This approach greatly reduces the range of operations variability. It makes the performance more predictable and easier to analyze.

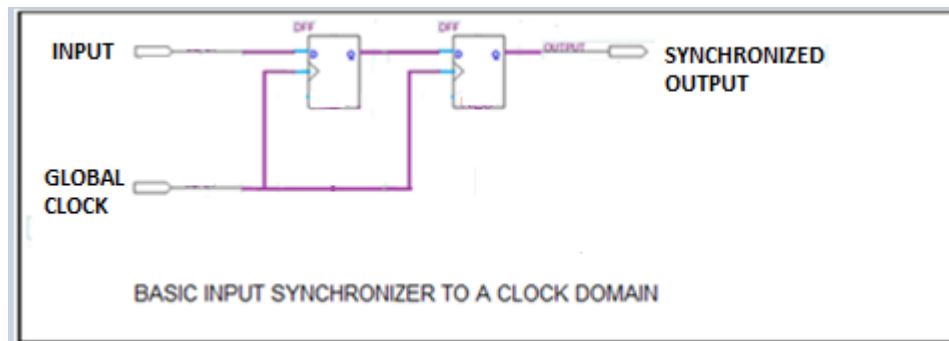


In synchronous designs, there is a Global clocking scheme used in a common clock domain. In the figure, one can see a typical array of registers used in a synchronous FPGA design. Between the SOURCE register on the left and the DESTINATION registers on the right, there can be a great deal of signal propagation delay along the paths through combinational logic and routing.

If it is deemed by the FPGA design tool, that moving data from one register output stage to another register input (on the same clock domain)) takes TOO LONG then the insertion of an intermediate clocked register pipeline stage is most likely required to be added.

However, someone may ask, “but external signals are NOT synchronized to this common global clock. They are **ASYNCHRONOUS** to the common global clock. How are these external signals supposed to be interfaced into a Synchronous Design?

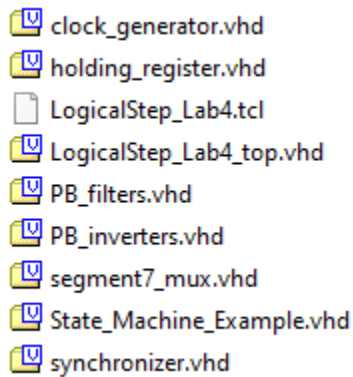
External Input Synchronization to the Global clock domain is done by adding two register stages in series (like a two-stage shift register) and clocked by the same common Global Clock.



If metastability occurs at the output of the first register, the metastability will not reach the output of the second stage and thereby prevent it from getting into the rest of the digital logic. Meanwhile the first stage output will have an entire Global clock cycle time for any metastable behavior to settle out before the second stage register transfers its input value from the first stage.

1.3.5 Initial Project Setup for Lab 4:

Start the Lab4 project like what was done in earlier Labs. Go to LEARN and download the Lab4 Zipped folder “Lab4” into the ECE-124 folder on your C: Drive. Extract the contents to create the new Lab4 project folder and its contents therein.



Your Lab4 Project Folder (downloaded from Learn) starts out like that shown at left.

There are three blocks that you are to use “as-is”. These are the segment7_mux, clock_generator and pb_filters files. The others you will be completing or developing.

You may copy the State_Machine_Example into another file for your own state machine design.

The pb_filters file is used to remove any “cross-talk noise glitches” on the pb_n inputs and rst_n.

The pb_inverters file has been changed to include an inverter for the rst_n input and rst output to the top-level design.

Start up the Intel Quartus Prime (v18.1) platform and begin a new project with the New Project Wizard. Enter the new Project parameters:

Project Folder: Lab4

Project Name: LogicalStep_Lab4

Project Top Level: LogicalStep_Lab4_top

Then click **NEXT** on the New Project Wizard Dialog Window. Then click **NEXT** again (with Empty Project).

Click FINISH on the New Project Wizard Dialog Window.

NEXT IMPORTANT STEP:

Run the Lab4 TCL script to assign the FPGA device type and pins for the LogicalStep FPGA.

Below are some screen shots of the Lab4 project starting files.

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY LogicalStep_Lab4_top IS
  PORT
  (
    clk_50      : in  std_logic;           -- The 50 MHz FPGA Clockinput
    rst_n       : in  std_logic;           -- The RESET input (ACTIVE LOW)
    pb_n        : in  std_logic_vector(3 downto 0); -- The push-button inputs (ACTIVE LOW)
    sw          : in  std_logic_vector(7 downto 0); -- The switch inputs
    leds        : out std_logic_vector(7 downto 0); -- for displaying the the lab4 project details

    -- you can add temporary output ports here if you need to debug your design
    -- or to add internal signals for your simulations
    -----

    seg7_data   : out std_logic_vector(6 downto 0); -- 7-bit outputs to a 7-segment
    seg7_char1   : out std_logic;                  -- seg7 digi selectors
    seg7_char2   : out std_logic;                  -- seg7 digi selectors
  );
END LogicalStep_Lab4_top;

```

Below is a screen shot of the LogicalStep_Lab4_top vhd file. You will finish the creation of the VHDL files for the synchronizers and holding registers and state machine later.

```

ARCHITECTURE SimpleCircuit OF LogicalStep_Lab4_top IS
  component segment7_mux port (
    clk      : in std_logic := '0';
    DIN2     : in std_logic_vector(6 downto 0); --bits 6 to 0 represent segments G,F,E,D,C,B,A
    DIN1     : in std_logic_vector(6 downto 0); --bits 6 to 0 represent segments G,F,E,D,C,B,A
    DOUT     : out std_logic_vector(6 downto 0);
    DIG2     : out std_logic;
    DIG1     : out std_logic;
  );
end component;

  component clock_generator port (
    sim_mode : in boolean;
    reset    : in std_logic;
    clk_in   : in std_logic;
    sm_clken : out std_logic;
    blink    : out std_logic;
  );
end component;

  component pb_filters port (
    clk_in : in std_logic;
    rst_n  : in std_logic;
    rst_n_filtered : out std_logic;
    pb_n   : in std_logic_vector(3 downto 0);
    pb_n_filtered : out std_logic_vector(3 downto 0);
  );
end component;

  component pb_inverters port (
    rst_n : in std_logic;
    rst   : out std_logic;
    pb_n_filtered : in std_logic_vector(3 downto 0);
    pb    : out std_logic_vector(3 downto 0);
  );
end component;

-- component synchronizer port(
--   clk      : in std_logic;
--   reset    : in std_logic;
--   din      : in std_logic;
--   dout     : out std_logic;
-- );
-- end component;
-- component holding_register port (
--   clk      : in std_logic;
--   reset    : in std_logic;
--   register_clr : in std_logic;
--   din      : in std_logic;
--   dout     : out std_logic;
-- );
-- end component;

  CONSTANT sim_mode      : boolean := FALSE; -- set to FALSE for LogicalStep board downloads
  SIGNAL rst, rst_n_filtered, synch_rst      : std_logic;
  SIGNAL sm_clken, blink_sig : std_logic;
  SIGNAL pb_n_filtered, pb : std_logic_vector(3 downto 0);

BEGIN
  INST0: pb_filters      port map (clk_in_50, rst_n, rst_n_filtered, pb_n, pb_n_filtered);
  INST1: pb_inverters    port map (rst_n_filtered, rst, pb_n_filtered, pb);
  INST2: synchronizer    port map (clk_in_50, synch_rst, rst, synch_rst); -- the synchronizer is also reset by synch_rst.
  INST3: clock_generator port map (sim_mode, synch_rst, clk_in_50, sm_clken, blink_sig);

END SimpleCircuit;

```

2.1 Project Brief for Lab4 (BACKGROUND DETAILS)

Today's Traffic Light Controllers can be considered as continuously running state machines. They go through their sequences in a very predictable and repeatable manner.

For policy (or legal) reasons it is often critical that Traffic Light Controller (TLC) signals be bounded to some type of accurate timing. For this lab, the TLC must change at about a 1Hz rate (ie: 1 cycle per 1000 msec) to enable the TLC to go through its sequence appropriately.

Another feature to be included for this project is for external pedestrian input signals to the TLC, to request changes to the traffic lights for crossing an intersection. Such requests are to be made "pending" until the appropriate time for switching.

This lab will use some style of a state machine design as the core design. As the state machine runs through its sequence, the appropriate outputs will then be used to illuminate the Traffic Lights being used in the intersection.

The state machine may be based on either a Moore or the Mealy type of state machine.

To eliminate all metastability issues with all sequential logic registers, the entire Lab4 Project design is required to be driven by a common Global Clock for a Synchronous Design approach.

The design work for Lab4 begins further down in the Lab4 Development Stages Section.

2.1.1 Traffic Light Controller Display on the LogicalStep Board

The LogicalStep board does not have any RED or AMBER LED's. So, the way that each column of traffic lights is to be shown on the board is on the the Seven-Segment Displays. They will be repurposed in a different kind of mode for this lab. We will no longer need the Seven-Segment decoder components. Instead, we will make use of just the 3 center segments of each display.

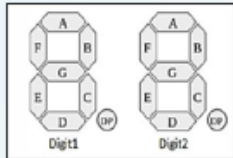
For each seven-segment display:



the BOTTOM segment (segment d) will represent a GREEN LIGHT on a Traffic Light column;
 the CENTER Segment (segment g) will represent an AMBER LIGHT on a Traffic Light column;
 the TOP Segment (segment a) will represent a RED Light on a Traffic Light column.

So, to activate the appropriate segment on a display the respective Segment7_mux segment input should be set to a '1'. You should be able to determine which inputs to use on the Segment7_mux for each seven-segment display (you can refer to Lab2

for those details). All other segments should be OFF in a Traffic Light column for this lab.

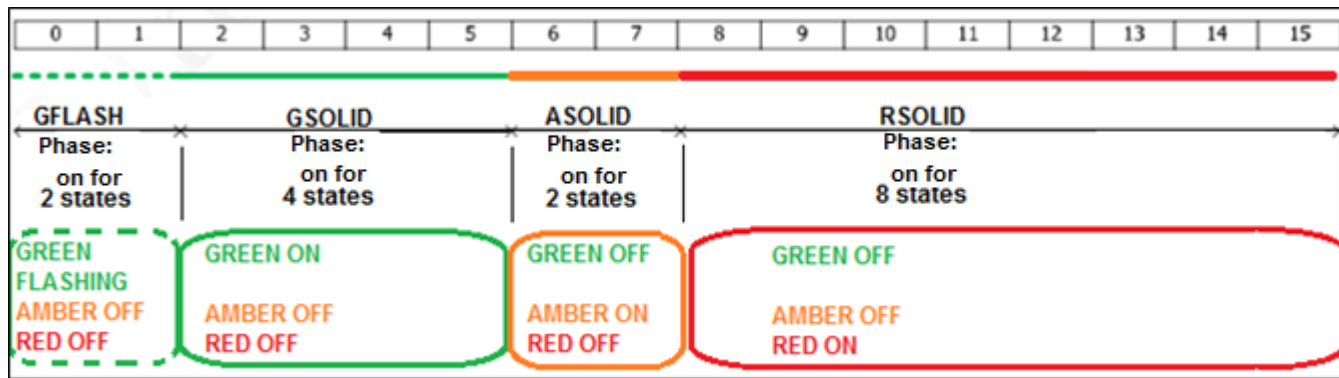


Your EW Traffic Light is to be displayed on Digit1 and NS on Digit2

2.1.2 Creating the Traffic Light Controller Sequencing

The **required** sequence for the TLC is shown below. The shows the behaviour for ONE direction of the TLC output is illustrated. The number of intervals for the TLC is shown as 16. The pattern starts with a flashing light on the GREEN for about 2 states followed by solid pattern for about 4 states. After that, the GREEN will be OFF, and the AMBER will be ON SOLID for about 2 states. Finally, the RED will be ON for 8 states while the other segments are OFF. After this, the SEQUENCE restarts.

For the FLASHING LIGHT states, you can use, as a source, the Blink_Sig signal connected to one of the Clock_Generator outputs.



In the above diagram one can see the different phases of the process.

You may construct your controller with either a Moore State Machine or a Mealy State Machine style. Either State Machine design must update to a new state only when it is enabled to do so at the next common 50 MHz Global Clock rising edge. The Enable signal may be connected to the Clock Generator "sm_clken". This signal is active for just ONE common 50 MHz Global Clock cycle at regular intervals.

Now, for the traffic intersection, the pattern for two TLC columns (NS and EW) is shown below.



NOTE: Each NS and EW roadway is just ONE way for this lab. We don't have enough Seg7 displays on the LogicalStep Board to implement for two-way traffic with Flashing GREEN operations etc.

In this example above, both of the TL columns can be decoded from a single set of outputs from one state machine. This is good for synchronization of the two TLC Column direction control.

You will be Implementing the above in your FPGA design (LogicalStep_Lab4_top) and then compile it and download it to observe its operation on the Logical Step board. The TOP Level file (LogicalStep_Lab4_top.vhd) must be in the STRUCTURAL VHDL style only. Signal concatenations at the top level are allowed.

Recall from Lab3, you can add any extra OUTPUT PORTS (for simulations only) to reveal any other internal nodes of interest for debugging purposes.

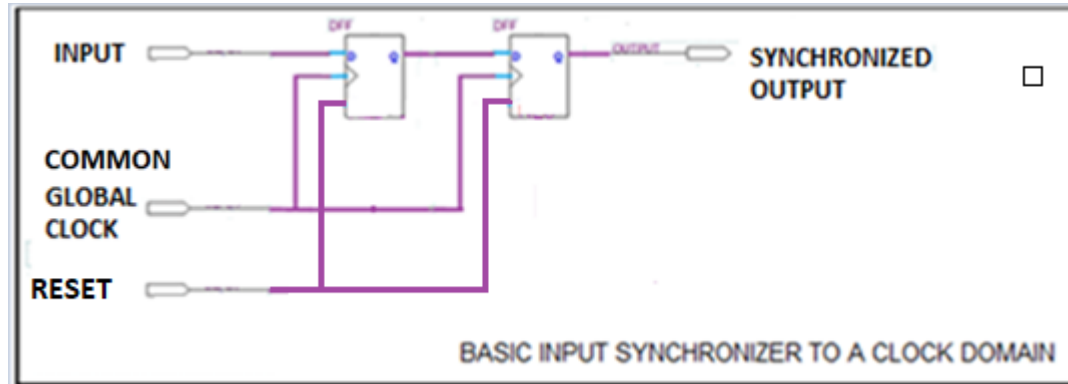
2.1.3 Adding Synchronized Inputs to the Traffic Light Controller

Now there will be a few enhancements added to the design. Some new system sensing inputs are to be provided to the state machine so that the Traffic Light system can accommodate new situations.

These new inputs will be from Pedestrian Crossing Request inputs for each of the two traffic directions. The State Machine sequence must change its sequencing of states in a **safe manner**.

There will be Pedestrian Crossing Request inputs for the NS direction and for the EW direction.

These inputs are ASYNCHRONOUS and must be synchronized and latched with the 50 MHz common Global clock (clkin_50).



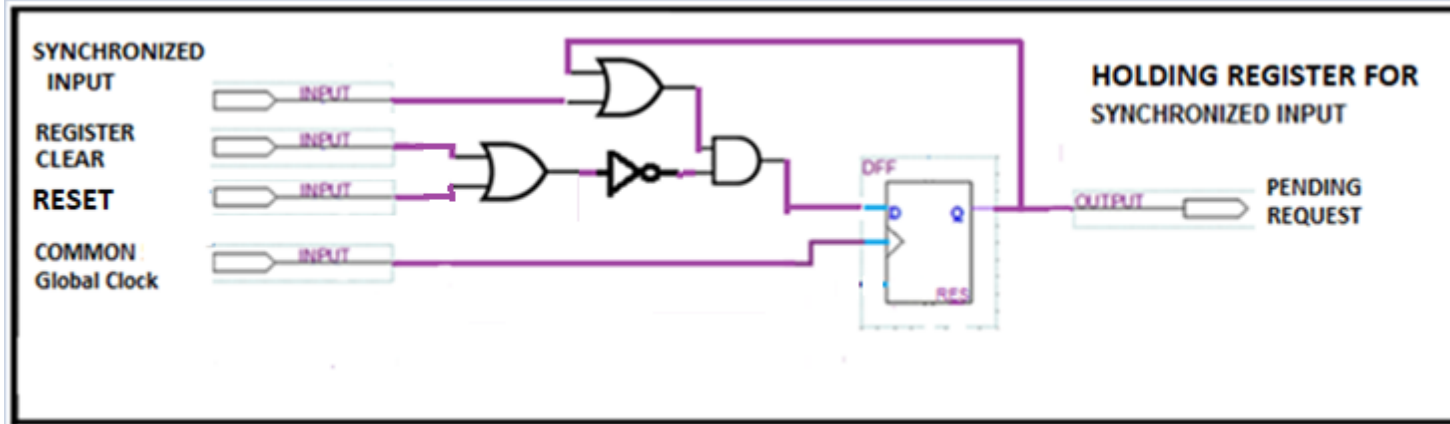
Complete a Basic Synchronizer VHDL design file with the functionality shown above.

The clock in in the Synchronizer instances will be connected to the 50MHz common Global Clock (clkin_50).

2.1.4 Adding Holding Registers for the Pedestrian Crossing Request

The Synchronizer outputs must be captured into a “holding register” for servicing by the state machine. When they are captured, the signals must stay in the ON state until the State Machine has acknowledged them and then cleared them. The captured or “held” signal for each direction is cleared by the state machine at an appropriate state in the sequence.

The Holding Register will be designed to receive the Synchronizer output and then hold it as a pending request output to the State Machine. When the State Machine reaches certain states in its sequence, it will accept the pending request, jump to new state, and then clear that holding register. (via the REGISTER CLEAR input signal shown below. The RESET signal (based on an inverted rst_n) is also to be used for this clearing function.



At the top level in the downloaded `Logical_Step_Lab4_top` file shown earlier, you can see that the Holding Register is already declared as a component. Complete the Holding Register design file with the logic, as shown above, in a separate file from the Synchronizer. Two of these functions are to be instantiated in the `LogicalStep_Lab4_top` design for these EW and NS Sensor Input pipelines.

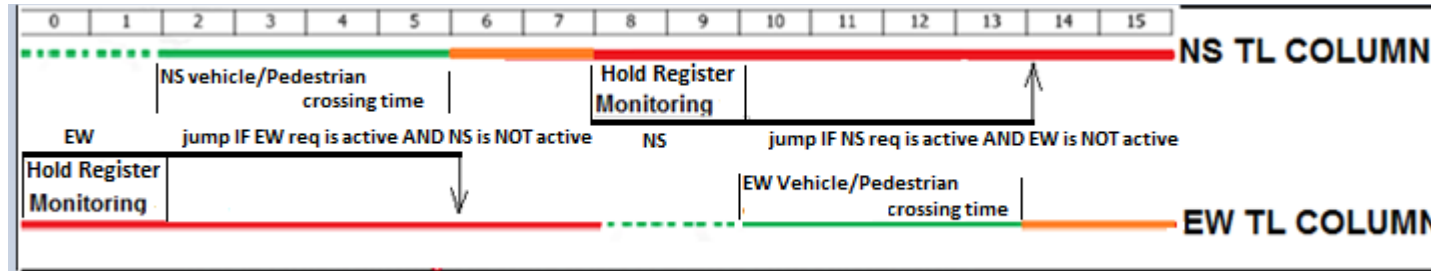
The clock port for each of the Holding Register instances will be connected to the common 50 MHz Global Clock (`clkin_50`). Each Register Clear input will come from the State Machine. The Register will clear at the next common 50 MHz Global Clock if the `Register_Clear` is active. The chip-wide RESET will also be brought into this function.

2.1.5 TLC State Machine Sequence Considerations for Pedestrian Crossing Requests

Now we need to look at where this new functionality can be inserted into the state machine sequencing. The Holding Register instance outputs are to be held until the State Machine reads the signal and then applies a Register Clear signal to the Holding Register instance at the right time. The key thing to remember is that the TLC State Machine **MUST NOT** cause the lights to change directly from GREEN TO RED for a NS or EW direction. An AMBER light **MUST** be active for a safe amount of time before the RED light is activated. Another constraint is that a Pedestrian Crossing Request **MUST NOT SHORTEN** the crossing time for pedestrians crossing during any GREEN light time.

The TLC State Machine can sense the activation of either pedestrian crossing request holding register, but the TLC must do a SAFE switching of the TLC lights in each of the two directions. A solution is to shorten the number of states where a RED light is active for a given direction when the pedestrian Crossing request is sensed.

A pedestrian wanting to cross in the NS direction will usually be facing a RED traffic light. A Pedestrian Crossing request can be activated, and the TLC State Machine will safely switch the lights by making sure that the EW direction gets to an AMBER traffic light state before it switches to a RED traffic light in the EW direction. This is so that the NS direction can get a GREEN traffic light SOONER and the pedestrian can cross in the NS direction. **But**, the other constraint for NOT SHORTENING the EW crossing time (i.e.: EW direction with the SOLID GREEN light) MUST BE observed.



For the above constraints, the state machine will monitor the NS Holding Register but only during States 8 and 9. If it is sensed as active during those states, then the State Machine transitions to State 14 so that the EW direction gets an AMBER light. The NS pedestrian will be able to cross when the NS direction gets a SOLID GREEN light. A LED indicator is to be used for the NS CROSSING display. It will be active when the SOLID GREEN light is active. **NOTE:** When the State Machine finally reaches **State 6** in the next TLC cycle, the NS Holding Register gets a REGISTER_CLEAR signal.

NOTE: for the NS Pedestrian Request, if the EW_Holding Register is active already, the state machine MUST NOT do the JUMP for the NS direction.

Similarly, the State Machine will also monitor the EW Holding Register but only during States 0 and 1. If it is sensed as active, then the State Machine will transition to State 6. The EW pedestrian will be able to cross when the EW direction gets a SOLID GREEN light. A LED indicator is to be used for the EW CROSSING display. It will be active when the SOLID GREEN light is active. **NOTE:** When the State Machine finally reaches **State 14** the EW Holding Register gets a REGISTER_CLEAR signal.

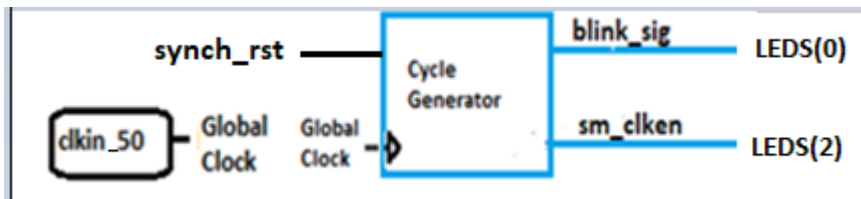
NOTE: for the EW Pedestrian Request, if the NS_Holding Register is active already, the state machine MUST NOT do the JUMP for the EW direction. This arrangement always ensures that the traffic control is done safely.

2.2 LAB4 DEVELOPMENT STAGES

For the following stages there may be temporary connections done with the pb_n's and the LEDs etc..

2.2.1 PART A:

- **Complete the design of the Synchronizer FIRST.** You can look for details back in Lab3 with the Bidr_Shift_Register example. The Synchronizer has an input and two DFF's and an output. The input to be synchronized to the clock is connected to the data input of the first DFF. There is also a RESET input.
- Starting with the LogicalStep_Lab4_top starting file, set the SIM_MODE Boolean constant to TRUE.
- From the Clock_Generator instance, add the sm_clken to LEDS (2) and blink_sig nets to LEDS (0) output ports for simulation observations (for temporary observation only).
- Run an Analysis/Synthesis compilation.
- Create a Waveform.vwf file to observe the simulation behaviour. You must add the Clkin_50, rst_n, and leds(2) and leds(0) nodes from the design. Set the Clkin_50 to run with a 20ns period over a 2 usec total simulation time.



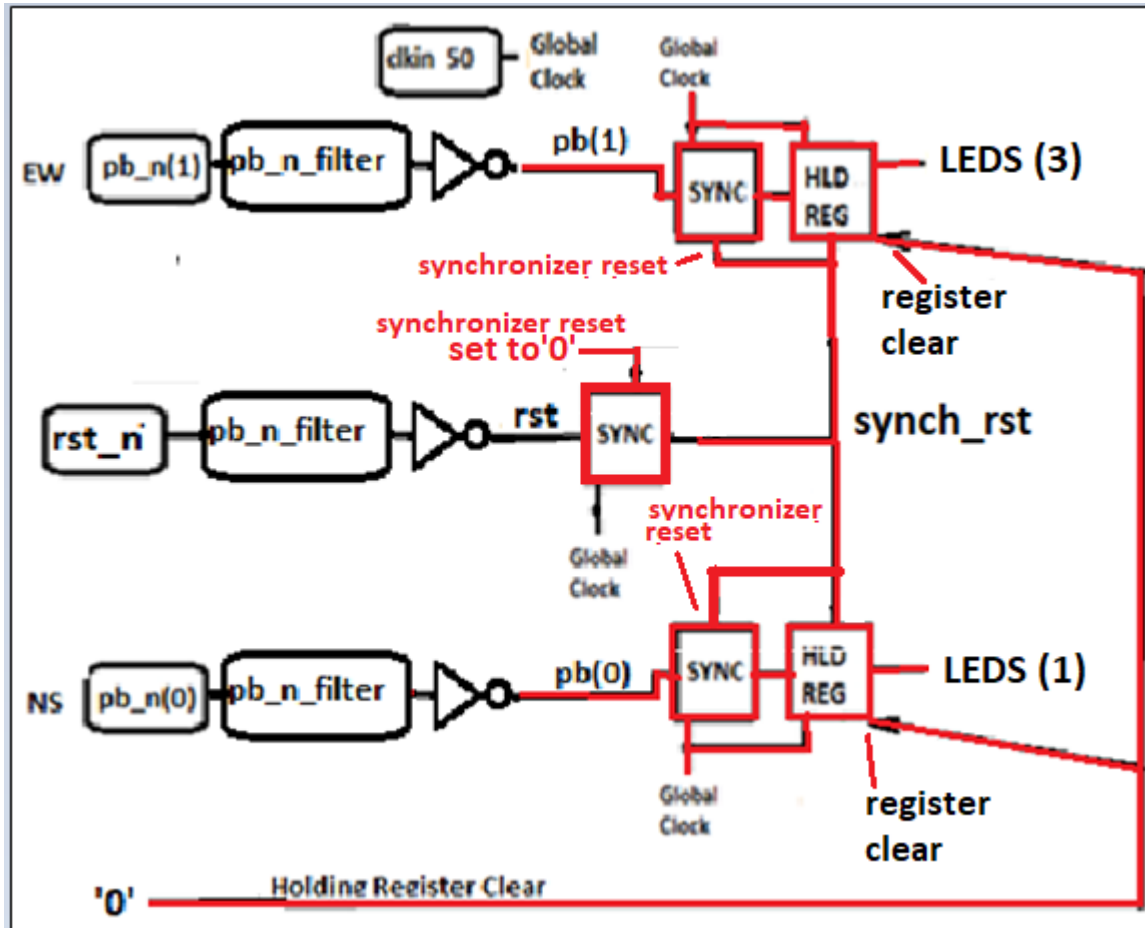
Remember that the rst_n input is ACTIVE-LOW. Set the rst_n signal to be active for four 50MHz clock cycles in duration at the beginning and then go to OFF.

Observe the timing of the clkin_50, sm_clken and blink_sig.

2.2.2 PART B:

- Set the SIM_MODE Boolean constant to TRUE. Use the **sketch** below to see the new connections to be made to the starting file of the LogicalStep_Lab4_top file.

-
- Complete the Holding_Register VHDL design and add two instances of it to the LogicalStep_Lab4_top file.
 - Connect the new Synchronizer outputs to the Synchronized inputs of the Holding Register instances.
 - Connect the Clkin_50 to the clock inputs of the Holding Register instances.
 - Connect the **synch_rst** signal (from the synchronizer used for the rst) only to the RESET inputs of the Holding Register instances and to the reset inputs of the instances for the pb(0) and pb(1) synchronizers. The Reset input to the rst Synchronizer will just connect to a '0' signal since that one won't be used.
 - Connect a '0' a temporary connection to the Holding Register instances for the Register_Clear inputs. We will just be using the **synch_rst** signal for resetting the Holding Register at this time.
 - Connect the Holding register instance outputs to LEDS(3) and to LEDs(1) as shown below.

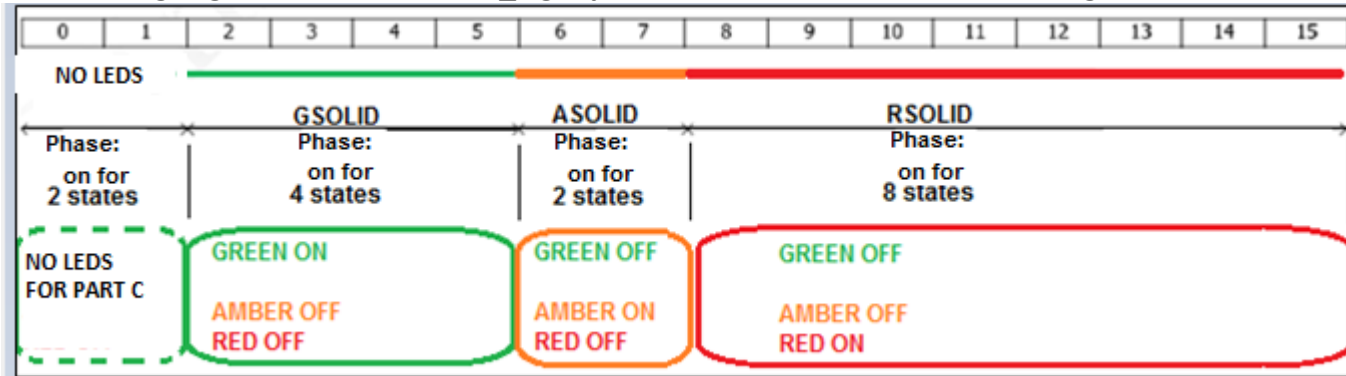


- Run an Analysis/Synthesis compilation and then create a Waveform.vwf file
- To see the simulation behaviour when the Clkin_50 is set to run with a 20ns period and with the pb_n(0), pb_n(1), rst_n inputs, activate the pb_n (1) or pb_n(0) inputs (**for at least three clk_50 cycles**) and observe that the holding register outputs (Leds(3) and LEDS(1)) are held active after the pb_n(1) or pb_n(0) buttons are released. Then, later on, activate the rst_n button and observe that the Holding registers get cleared on LEDs (3) and LEDS(1).

- **Take a screen shot of this simulation and keep it for your Lab4 Report.**
- Set the SIM_MODE Boolean constant to FALSE again and do a FULL Compilation and download the “.sof” file to the LogicalStep board.
- Observe that a pb_n(1) and pb_n(0) activate the Holding Register outputs on one of LEDS (3) or (1) on the LogicalStep board.

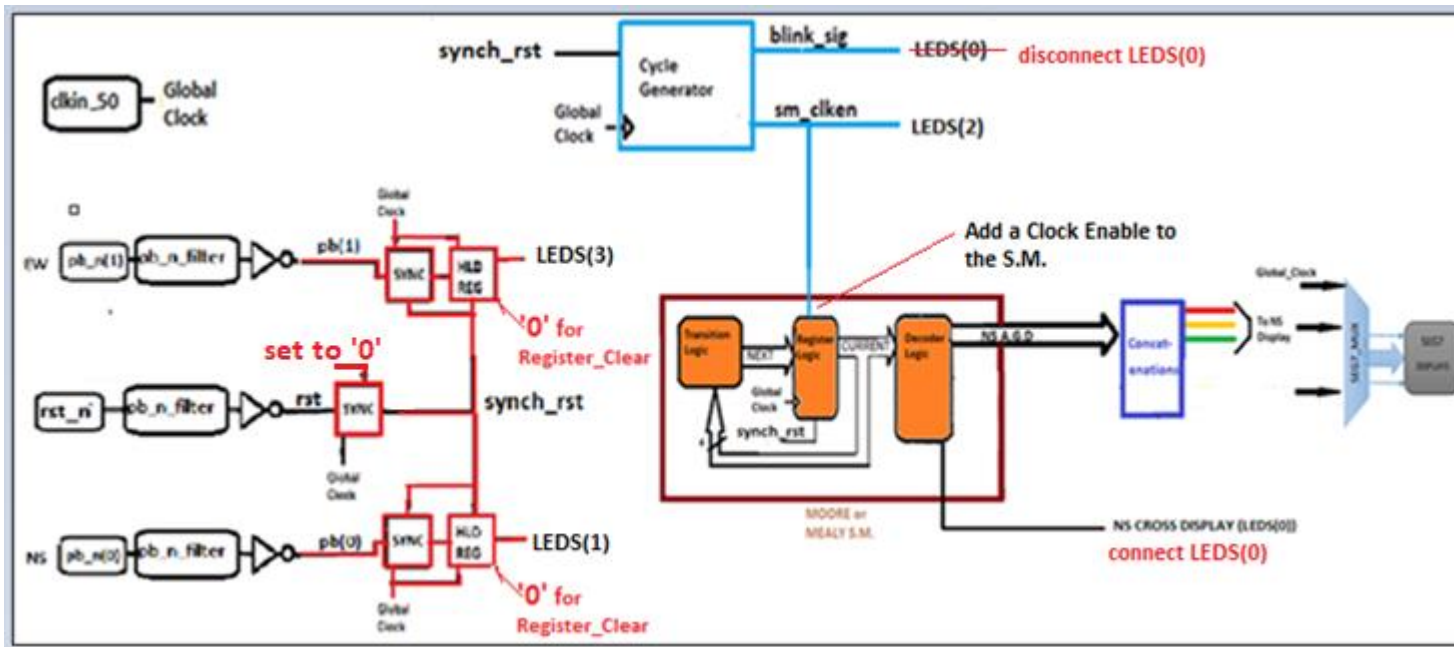
2.2.3 PART C:

- Add the synch_rst signal to the State_Machine RESET input.
- Develop the TLC State Machine (Moore or Mealy form) with the state sequence described earlier for the NS direction first.
Remember to add the sm_clken enable signal to the TLC State Machine Register section.
- Make the Transition Logic section (COMBINATIONAL LOGIC ONLY) just go through the sequence of 16 states (ie: S0—S1—S2—S3—S4—S5—S6—S7—S8—S9—S10—S11—S12—S13--S14—S15—S0—S1 etc.).
- **The holding registers and the blink_sig inputs to the State Machine can be Ignored for now. NO LEDS in S0, S1 states for Part C.**

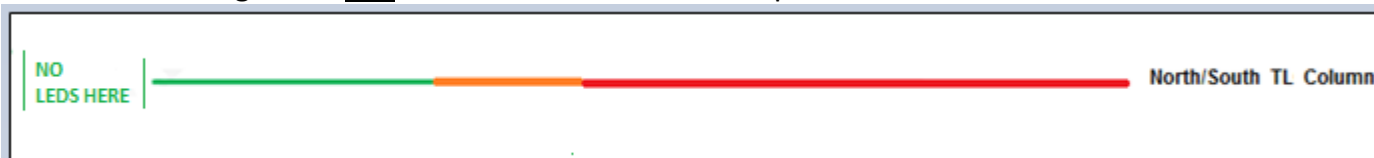


- Then add outputs into the Decoder section (COMBINATIONAL LOGIC ONLY) to drive the LEDS for the NS Traffic light column (segments A, G and D only, the other segments must be kept OFF).
- Declare a component for the TLC State Machine file in the top-level file and add an instance of it as well at the top level.

- Use concatenation to drive all segment values to the Segment7_Mux instance for Dig2 to show the NS TLC activity. The NS_Crossing Display Signal must be active ONLY when the NS lights are a SOLID GREEN. Disconnect LEDS(0) from the “blink_sig” signal used in Part A and then add the NS_CROSSING_DISPLAY signal to LEDS(0).

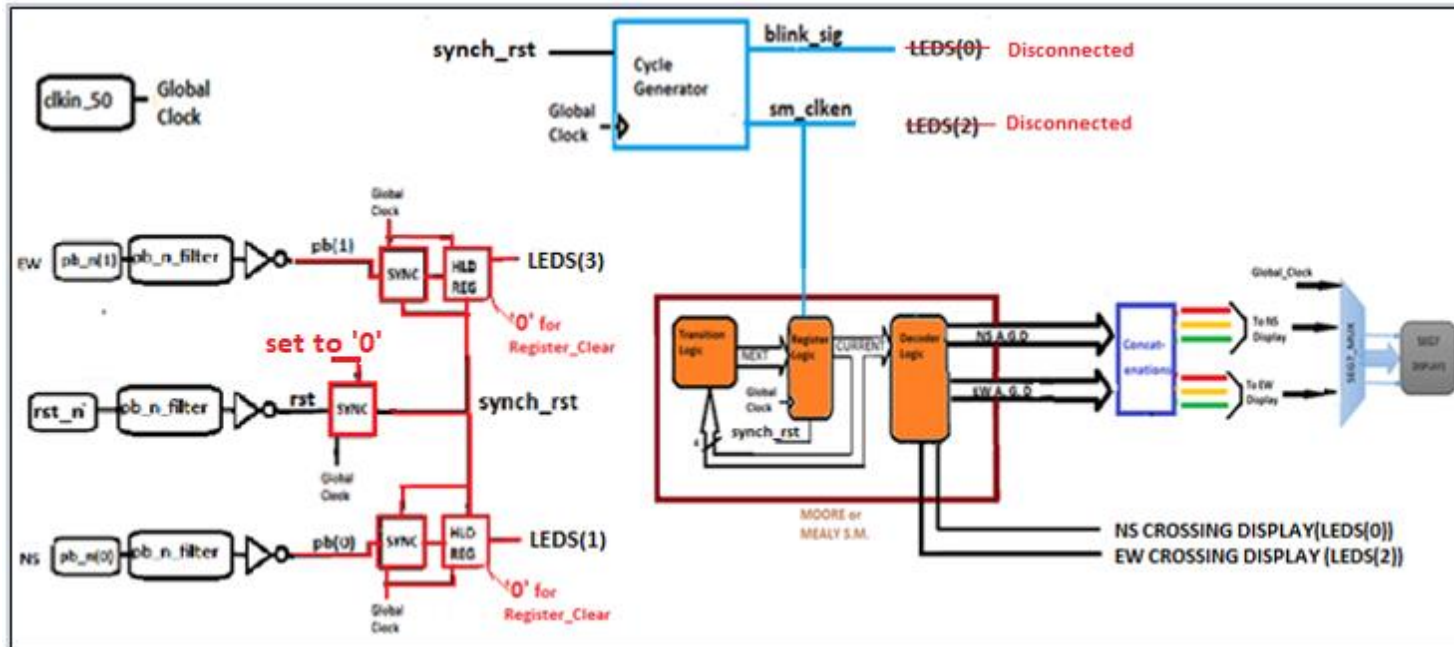


- Do a FULL COMPILE (again with the SIM_MODE set to FALSE) and test it on the LogicalStep board to observe the pseudo- traffic light sequence for the NS direction (on DIGIT 2).
- The NS GREEN Light will **not** have a FLASHING GREEN operation in states S0 and S1 at this time.



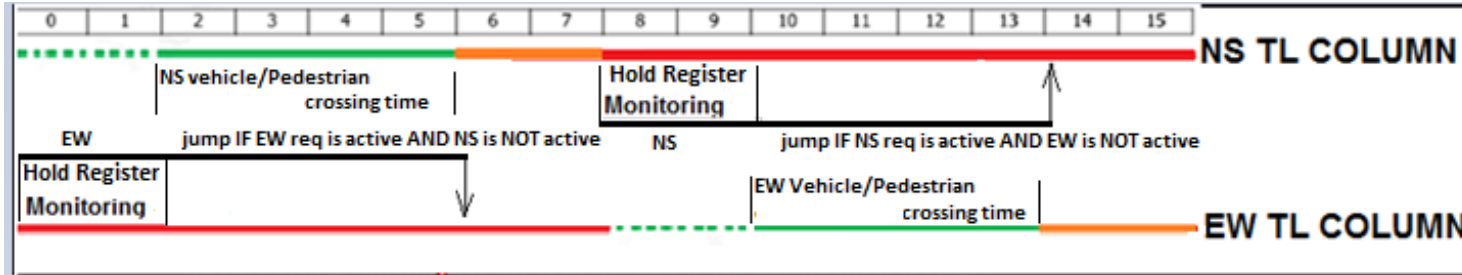
2.2.4 PART D:

- Modify the TLC State Machine design by **adding** outputs to the Decoder section to drive the LEDs for the EW Traffic light column (segments A, G and D only, the other segments must be kept OFF).
- Use concatenation to drive all segment values to the Segment7_Mux instance for Dig1 to show the EW TLC activity. The EW_Crossing Display Signal must be active ONLY when the EW lights are a SOLID GREEN. Disconnect Leds(2) from the "sm_clken signal used in Part A and then add the EW_CROSSING DISPLAY signal to Leds(2)
- Do a FULL COMPILE (again with the SIM_MODE set to FALSE) and test it on the LogicalStep board to observe the new traffic light sequence for the NS direction (on DIGIT 2) & the EW direction (on DIGIT 1).
- The NS and EW GREEN Lights will **not** have a FLASHING GREEN operation at this time.

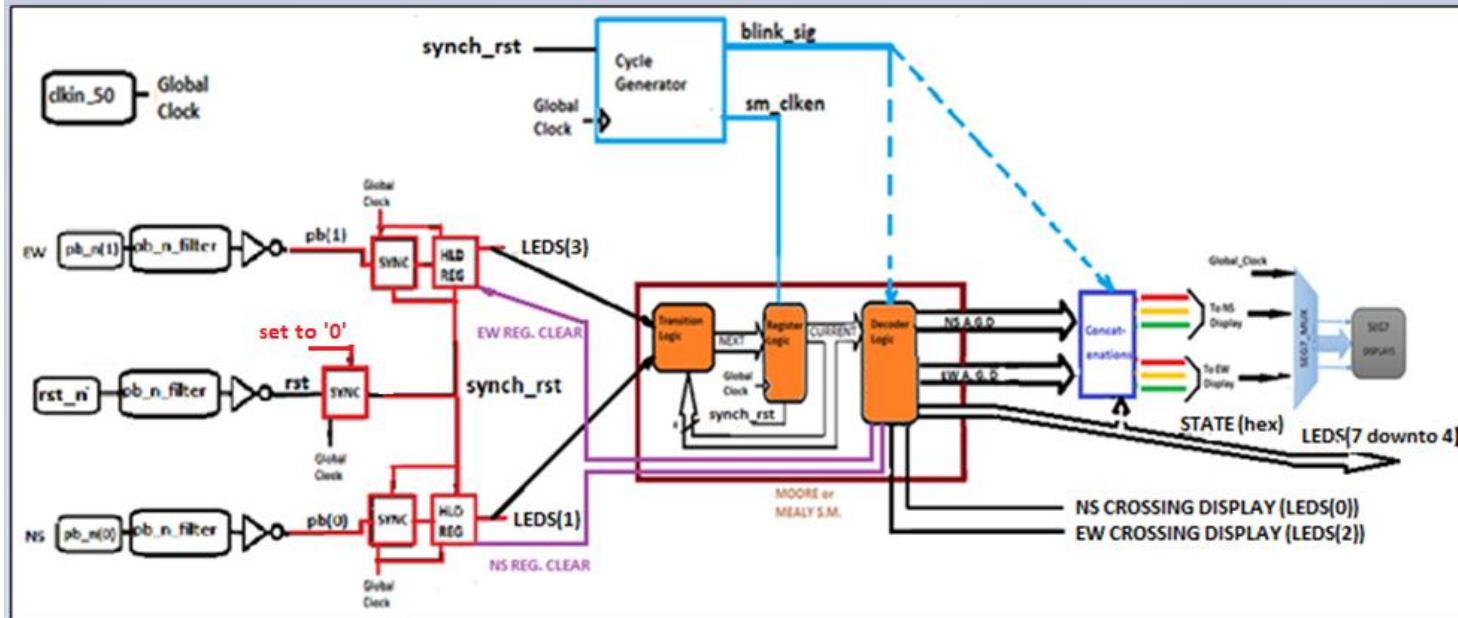


2.2.5 PART E:

- Add the Holding Register instance outputs (that are connected to LEDS(3, 2)) also connect to the appropriate inputs of the TLC State machine instance (that connect to the Transition section inputs).



- Modify the TLC State Machine Transition Section to sense the activation of the Holding Register inputs at the appropriate sensing states described earlier for each direction, and then have the TLC state machine jump to the APPROPRIATE state, shown above. Do NOT make the Jump if the Holding Reg Input for the other direction is already ACTIVE. Activate the NS or EW Reg Clear signals to the appropriate Holding Register instance.
- Add a 4-bit State Number to the Decoder output for displaying on LEDs (7 down to 4).
- You may add the “blink_sig” to either the state machine Decoder as an INPUT or to the concatenation block as an input. This signal will be used to FLASH the Green lights at the appropriate states. Alternatively, if the “blink sig” is added to the concatenation block then you may have to add the STATE NUMBER signals to the Concatenation block as well.



2.2.1 PART F:

- Set the SIM_MODE to **TRUE** again and then run an Analysis/Synthesis compilation.
- Create/modify a Waveform.vwf file to include the Input and output pins from the Table below.
- Set the End Time to 32 usec.
- Perform a simulation with activity from the following signals (**you will have to add some of them as output ports at the top-level file so that you can see them in simulation**):

INPUTS and Internal Signals:

clk_in_50, sm_clk_en, blink_sig, pb_n(0), pb_n(1), rst_n, segments a,d and g for each of NS and EW,

OUTPUTS:

LEDS(7 downto 4) for the TLC states; and LEDS(3downto 0) for
leds(3) <= EW_req; from the EW holding register output
leds(2) <= EW Crossing Display
leds(1) <= NS_req; from the NS holding register output
leds(0) <= NS Crossing Display

Set the END TIME for the simulation to be 32 usec. Set the signals to run with a 20 ns clkin_50 signal. The TLC State Machine must only advance on a rising edge of clkin_50 when the sm_clken is active.

USE THE ZOOM Feature in the Simulator window to place your pb(1), pb(0) stimulus pulses at the appropriate times mentioned below..

Look for why the second crossing request does NOT activate a JUMP in the state machine-sequence. Add this as an annotation in your student simulation with pointer arrows.

PEDESTRIAN INPUTS:

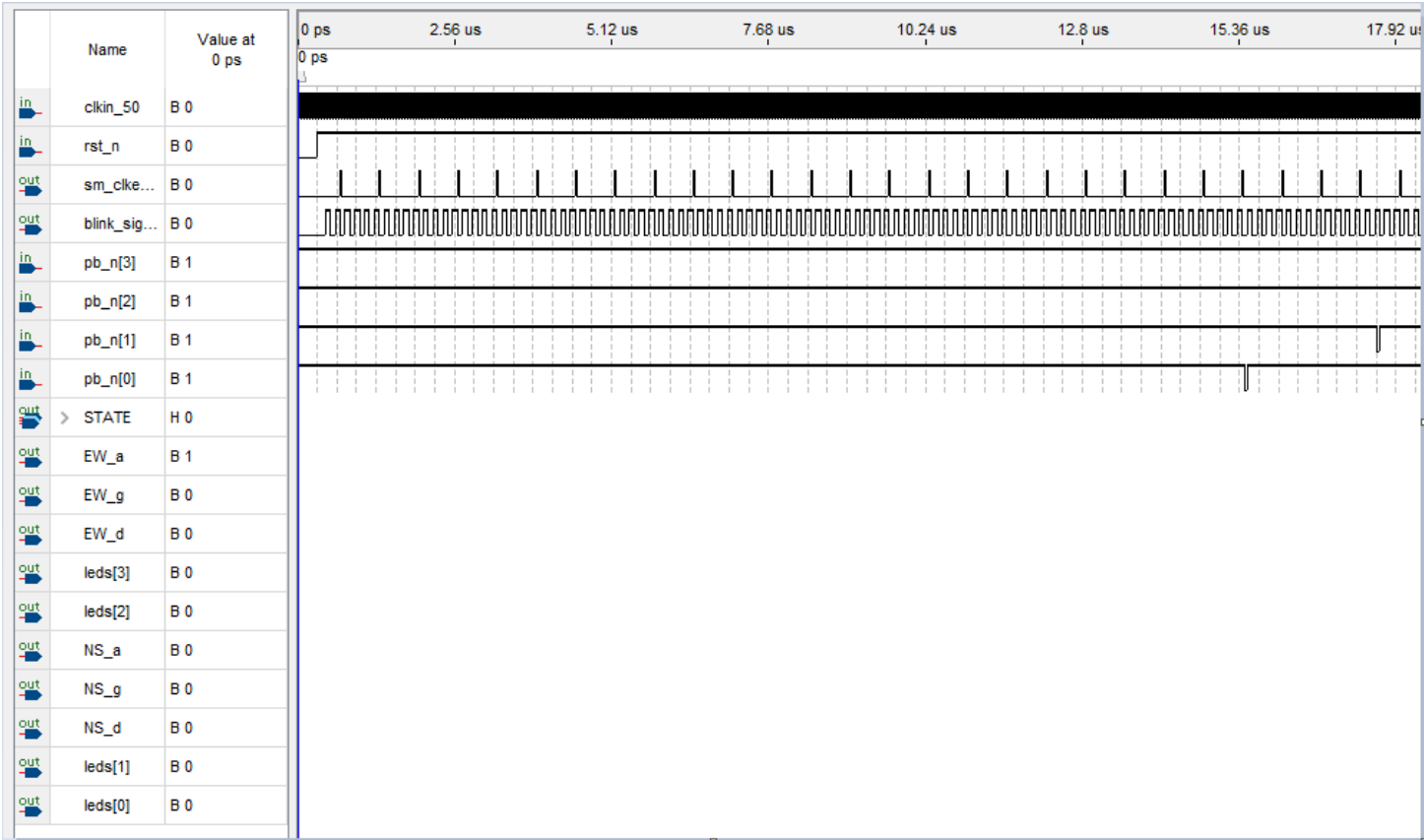
Apply a four 50MHz clock cycle-wide pulse to the rst_n input at the beginning.

The NS Pedestrian Crossing Request Button (pb_n(0)), should activate at the 15.6 usec point for at least THREE clkin_50 clock cycles. It should get sensed by the state machine and then a state jump change by the state machine should be evident in the simulation. The request should then get cleared according to the earlier description above.

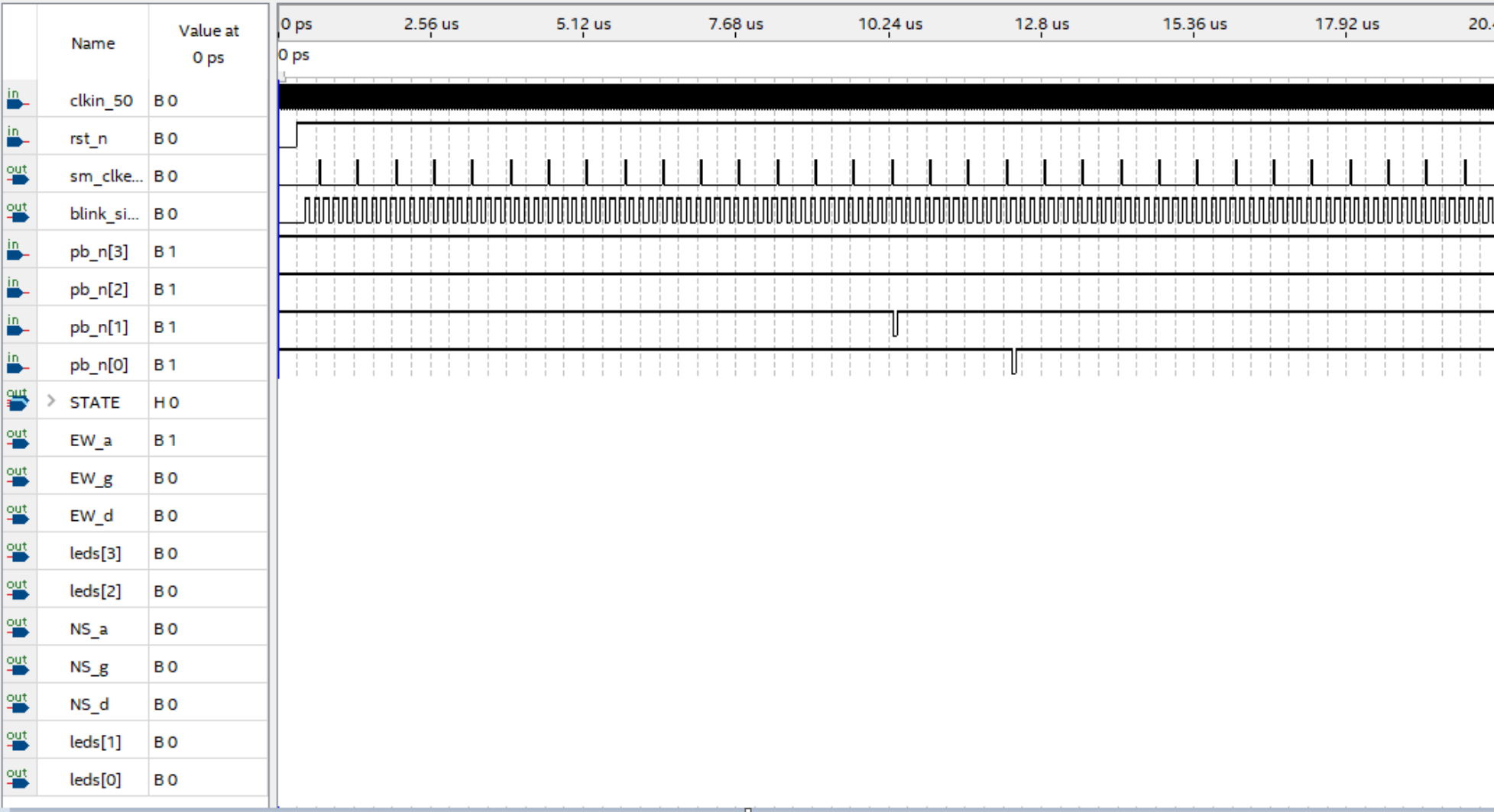
The EW Pedestrian Crossing Request (pb_n(1)), should activate in another sequence at the 17.6 usec point for at least THREE clkin_50 clock cycles.

Any jump change by the state machine should be evident in the simulation. The request should then get cleared. **Take a screen shot of this simulation and keep it for your Lab4 Report.**

The STIMULUS should look something like the following but run out for a full 32 usec.



The Student2 STIMULUS should look something like the following but run out for a full 32 usec

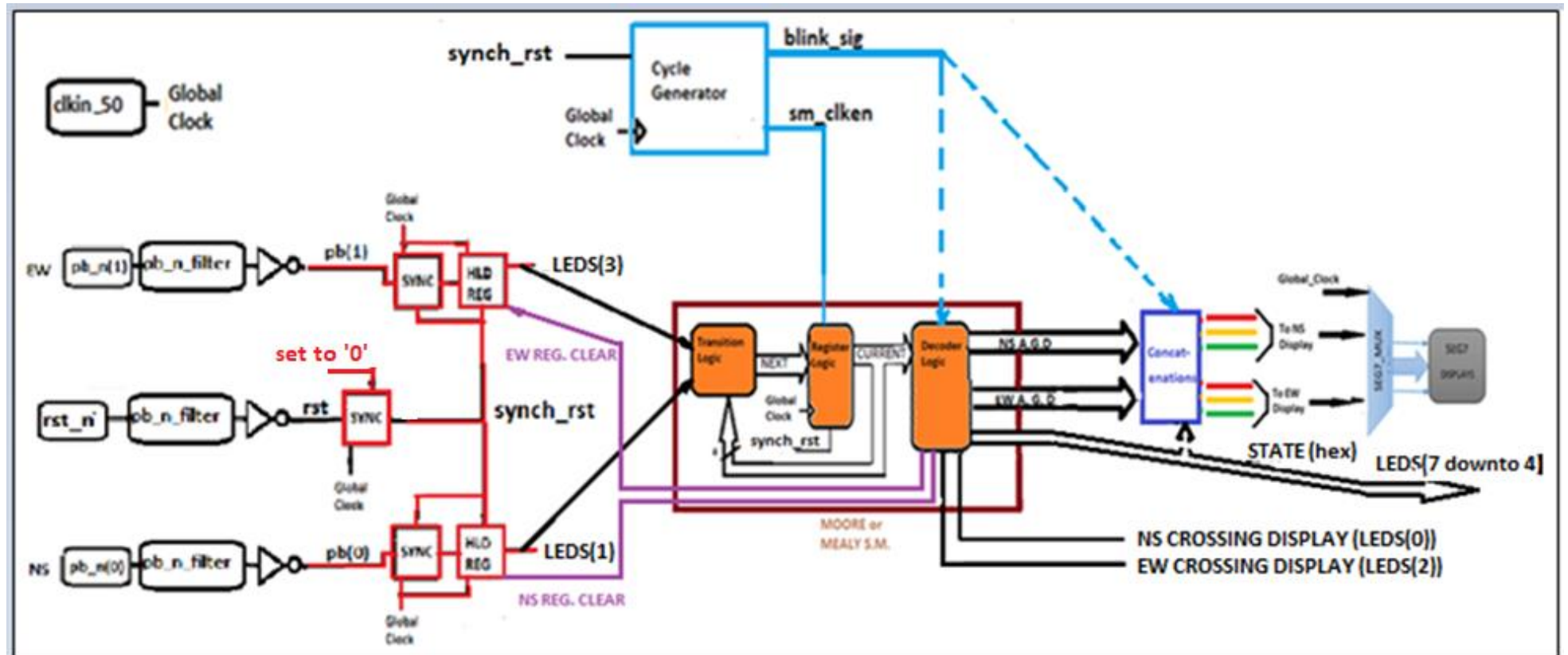


2.2.2 PART G:

- Set the SIM_MODE to FALSE.
- Comment out any of the extra Output Ports used for simulation of internal signals (e.g.:sm_clken, blink_sig etc.)
- Do a Full Compile, download the config file to the LogicalStep board to confirm the functionality.

FOR THE FINAL DEMO DESIGN MAKE SURE THAT THE LAB4 PROJECT INPUTS AND OUTPUTS ARE ALIGNED WITH THE TABLE BELOW.

SIGNAL TYPE:	SIGNAL Function:	ASSIGNED PORT(s):	Comment
Inputs	clkin_50	Clkin_50	50 MHz input clock
	RESET	Rst_n	RESET to clear State Machine and registers
	EW Pedestrian Crossing Request	Pb_n(1)	input for EW crossing. It must be inverted for synchronizer input
	NS Pedestrian Crossing Request	Pb_n(0)	input for NS crossing. It must be inverted for synchronizer input
Outputs	seg7_data [for segments A,D,G]	Seg7_data[A,D,G]	Used to illuminate Seg7 LED display for TLC Columns
	seg7_data [for segments B,C,E,F]	Seg7_data[B,C,E,F]	OFF
	seg7_char1,	seg7_char1,	Driven by seg7_mux for EW TL Column Digit1 <-NOTE
	seg7_char2	seg7_char2	Driven by seg7_mux for NS TL Column Digit2<-NOTE
	Holding Register – EW EW CROSS DISPLAY Holding Register - NS NS CROSS DISPLAY	leds[3], leds[2], leds[1], leds[0]	Pending EW Crossing Request EW Crossing Display Pending NS Crossing Request NS Crossing Display
	State Number[3..0]	Leds[7 downto 4]	State Machine State Number



2.2.3 PART H: STATE DIAGRAM

- create a State Diagram with a State Table (from Quartus Tools>> Netlist Viewers>>State Machine Viewer). Create a separate screen shot of the State Transition Table
- The State Diagram must show all 16 states of the state machine and the transitions between them.

Keep these screen shots of the State Diagram and Transition Table and keep them for your Lab4 Report.

2.3 LAB4 Report Submission Requirements

- ONE pdf report file (in PDF format with Portrait or Landscape orientation) **for each group.**
- Screenshots of ALL VHDL files (except the clock generator, pb filters, Segment7 mux) with Team member name(s) at the top of each file. The TOP level is to be in **Structural VHDL format only**. Signal concatenations are allowed at the top level as well. Code comments are required. See the Lab4 Report Rubric on Learn.
- Simulation screen shot of the Synchronizer and Holding Register (from Part B) with text font annotations.
- Simulation screen shot of the TLC State Machine from Part E (must include the clkin_50, sm_clken and blink_sig signals, the TLC segments (a,d,g for the NS and EW trafficlighs), State Machine State Number, Holding Register outputs for NS/ EW Request signals and EW/NS Pedestrian Crossing indicators. All of these things are to have text font annotations.
- State Machine State Diagram. The State Transitions Table.